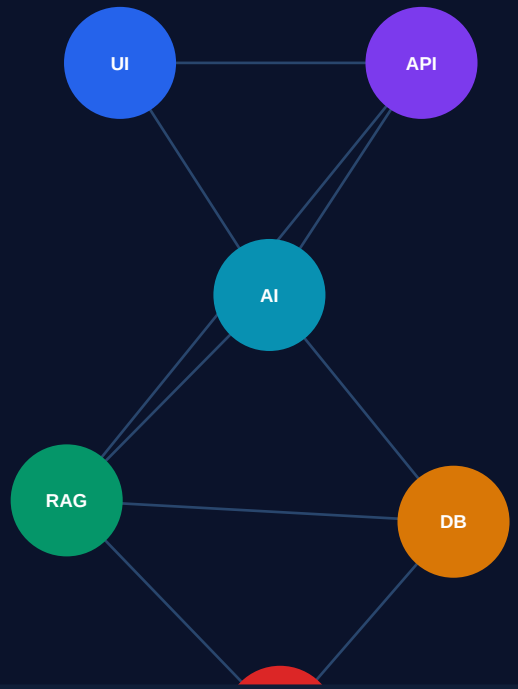
INTERVIEW WORKFLOW GUIDE

LawShield

End-to-End Working Flow

A clear code-derived walkthrough of how authentication, document ingestion, PII protection, memory, LangGraph routing, Self-RAG, SSE streaming and persistence work together.



8-PAGE ROADMAP

01 System overview
02 Document ingestion
03 Chat + privacy
04 Agent routing
05 Self-RAG
06 SSE + persistence
07 Interview summary

The complete application working flow

One durable request path from the browser to validated AI output

THREE PERSISTENCE ROLES

Durable truth

PostgreSQL

- users and JWT identity
- threads and messages
- facts and document metadata

Semantic evidence

Qdrant

- document embeddings
- conversation vectors
- cosine similarity retrieval

Lexical evidence

Elasticsearch

- chunk text and metadata
- BM25 keyword ranking
- optional TF fallback

Operational contract

Frontend

renders UI; never calls model vendors

Backend

authenticates ownership and writes durable records

AI service

masks, routes, retrieves, evaluates and restores

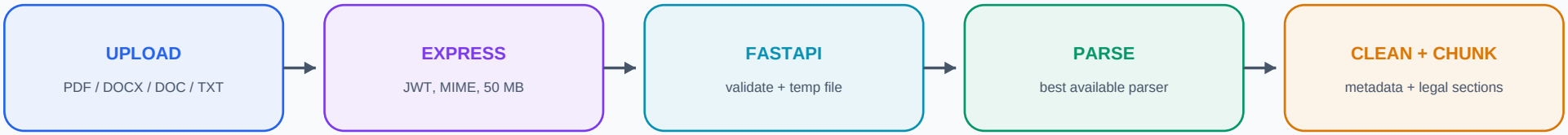
Safety boundary

only validated output is returned and persisted

Sources: README.md; backend/server.js; ai_service/main.py; chat/chat_service.py

Document upload and dual-index ingestion

Best-available parsing feeds semantic and lexical retrieval stores

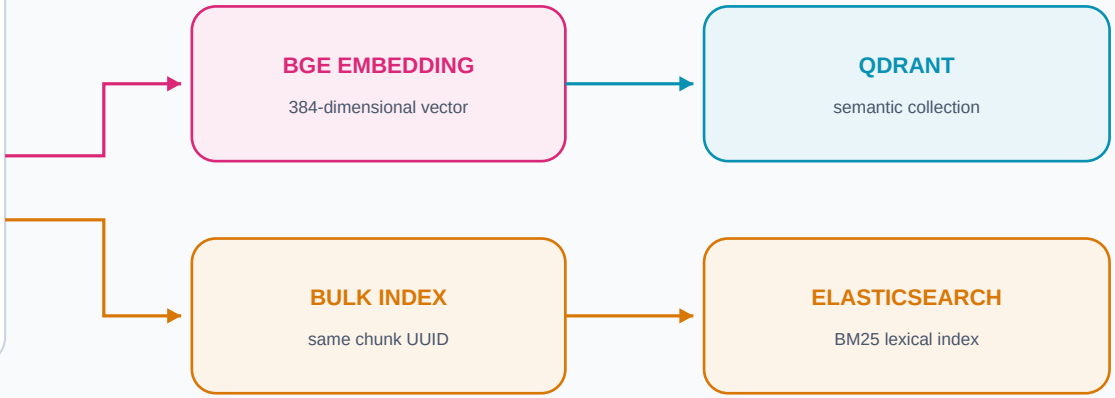


PDF FALLBACK WATERFALL

same UUID

Shared chunk payload

- page_content: normalized legal text
- metadata: filename, section title and number
- Qdrant payload receives vector plus text
- Elasticsearch receives text plus metadata



Sources: backend/routes/documents.js; document_processing/loaders.py; rag/pipelines.py

Chat preparation, privacy and memory

Durable history becomes a masked, memory-enriched LangGraph state

before model

PII protection

- Names, email, phone, organization and location
- Aadhaar, PAN, GSTIN, passport, IFSC and bank details
- Stable placeholders such as <PERSON_1>
- Original mapping remains local for restoration

read path

Prompt memory

- Backend history is the durable source of truth
- Sliding window is rebuilt on every request
- Long-term facts come from PostgreSQL
- Related exchanges come from Qdrant

LangGraph input

AgentState

- messages append through an operator.add reducer
- routing, research and specialist fields start blank
- nodes return only fields they changed
- terminal AIMessage becomes the answer

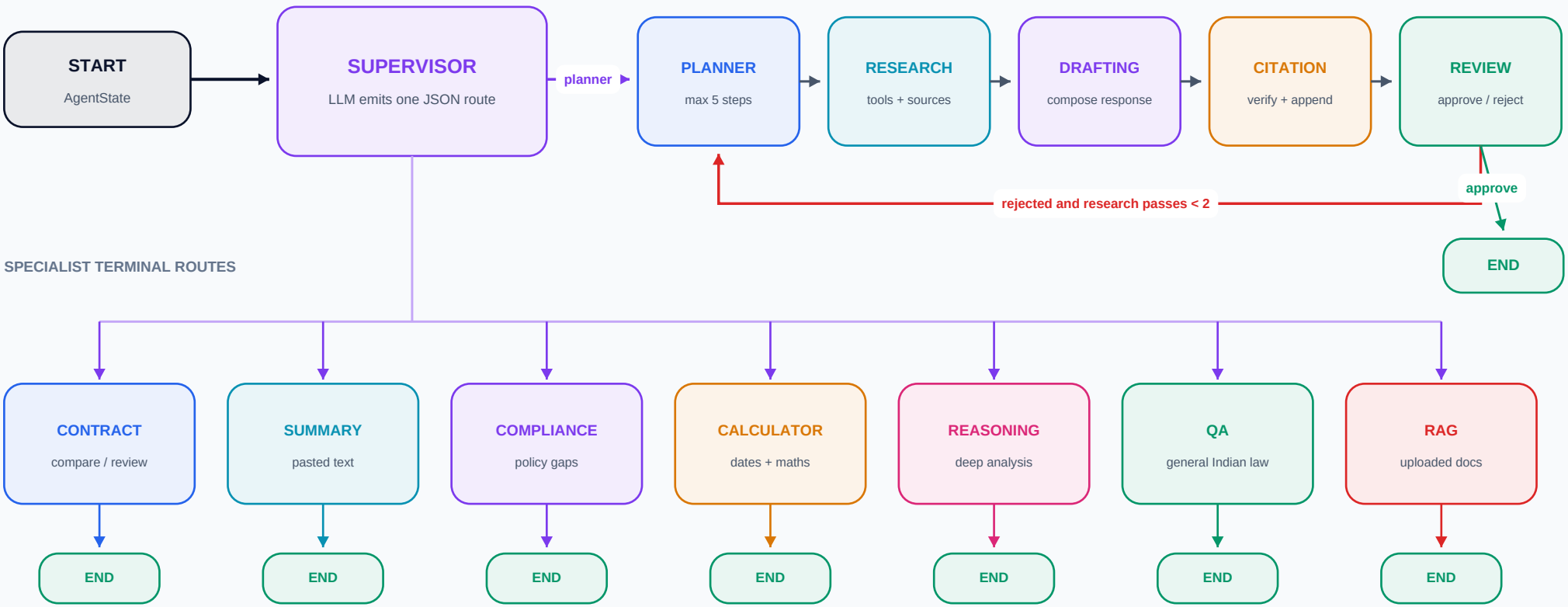
Privacy boundary: raw user PII is replaced before any LangGraph or model invocation. Output validation occurs before the placeholder map restores original values.

Latency boundary: embedding search and background memory writes run through asyncio.to_thread so blocking model or database work does not freeze FastAPI's event loop.

Sources: backend/routes/chat.js; chat/chat_service.py; masking/*; memory/*

Supervisor routing and agent execution

One classifier selects either a specialist or the iterative research pipeline



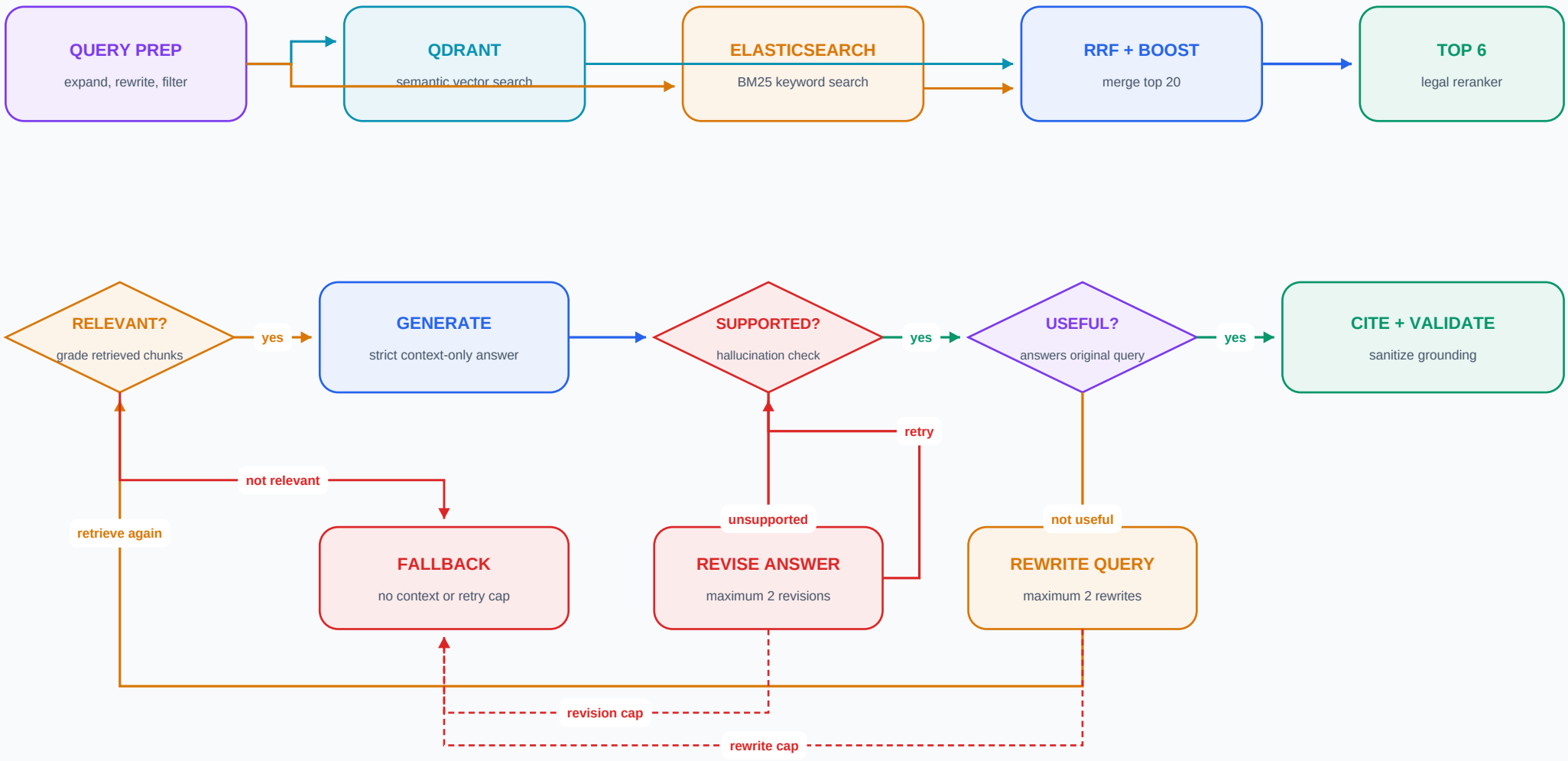
Routing examples

RAG	QA	PLANNER	REASONING
What is the notice period in my uploaded contract?	What is habeas corpus?	Draft a cease-and-desist letter.	Analyze this pasted clause for risk.

Sources: graph/graph.py; graph/conditions/routing.py; agents/supervisor/agent.py

Hybrid retrieval and bounded answer repair

Semantic plus keyword evidence passes relevance, grounding and usefulness gates

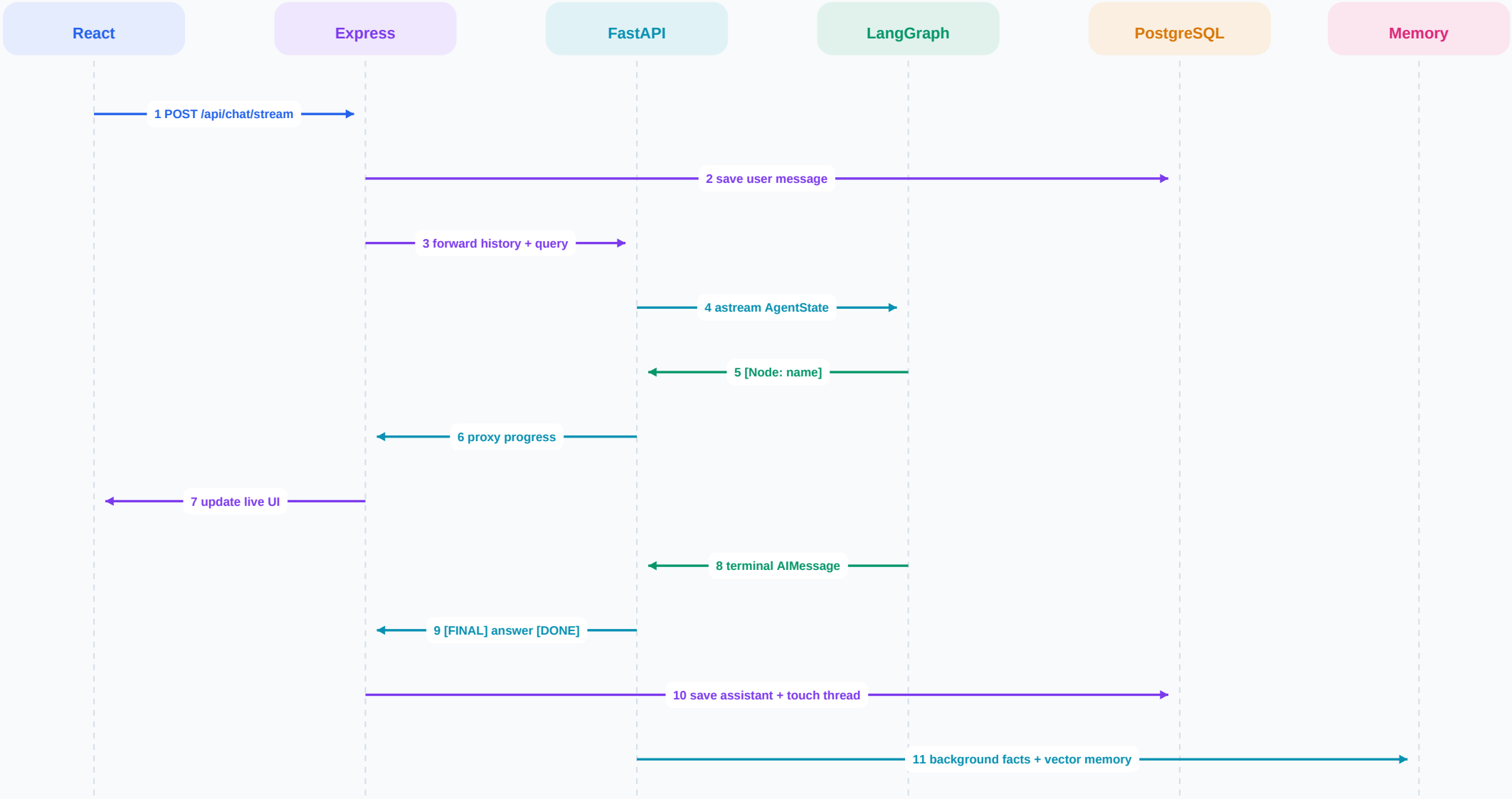


Maximum work per query: initial retrieval plus 2 rewrites, and initial answer plus 2 revisions. Exhausted checks route to fallback instead of looping forever.

Sources: rag/hybrid/search.py; rag/reranker.py; rag/self_rag/graph.py

SSE streaming, validation and persistence

Progress is streamed immediately; durable state is committed around the AI call



The user message is written before generation. If the AI call fails, the question remains in the thread and the backend can return a controlled error response.

After output validation and PII restoration, the assistant message is durable. Fact extraction and vector-memory writes are independent background tasks.

Sources: backend/routes/chat.js; chat/router.py; chat/chat_service.py; memory/*

How to explain LawShield in an interview

say this

90-second explanation

LawShield uses React for the workspace, Express as an authenticated API gateway and FastAPI for AI orchestration. Uploaded documents are parsed and dual-indexed into Qdrant for semantic retrieval and Elasticsearch for BM25 keyword retrieval. Before a query reaches the model, Presidio and custom Indian recognizers mask PII. A LangGraph supervisor routes the request to the correct specialist or a bounded research pipeline. Document questions use Self-RAG relevance, grounding and usefulness checks. The validated answer is restored, streamed with SSE and persisted in PostgreSQL.

Grounded answers

Self-RAG, citations and output validation reduce unsupported legal claims.

Hybrid retrieval

Qdrant meaning plus Elasticsearch exact legal terminology.

Privacy first

PII is replaced before any model invocation and restored later.

Bounded systems

Revision, rewrite and research loops have explicit caps.

mention honestly

Current limitations

- Deleting SQL metadata does not remove Qdrant or Elasticsearch chunks
- Chunks need document_id and user_id for tenant-safe filtering
- Elasticsearch is optional and security is disabled in local Docker

improve

Next production steps

- Transactional or outbox-driven cross-store deletion
- Per-user metadata filters in every retrieval request
- Health checks, secured Elasticsearch and observable ingestion status

Sources: project architecture and implementation notes